Programming 1

Orga / Recap

- We all have a solid understanding of
  - variables, types and collections?
  - Loops, conditions
  - Functions & modules
  - Object oriented software development
  - UI / Tkinter
- Any questions so far?

Orga: Eval AIN

Programming 1 (AIN-B):

https://eva.th-deg.de/evasys/online.php?typ=html&user_tan=ZKZ7J

Tkinter & Canvas recap

- To create shapes use:
  - canvas.create_<valid shape name>(….)
- To move an item:
  - canvas.move(item, x, y)
  - (*) canvas.bbox(item) returns a bounding box (x1,y1,x2,y2) – Useful to perform collision checks

Calendar

6./7.10.	Intro, Orga, Hello, World!	8./9.12.	lectures
13./14.10.	Variables, basics	15./16.12.	mid-term
20.21./10.	Collections, Lists, ...	22./23.12.	lectures
27./28.10.	Conditions and Loops	12./13.1.	lectures
3./4.11.	Functions and Modules	18./19.1.	recap
10./11.11.	Classes and Methods	from 26.1.	exams...
17./18.11.	UI + TK basics		
24./25.11.	Invaders and Collisions		
01./02.12.	Code structure and game		

SpaceShips

Spaceship status

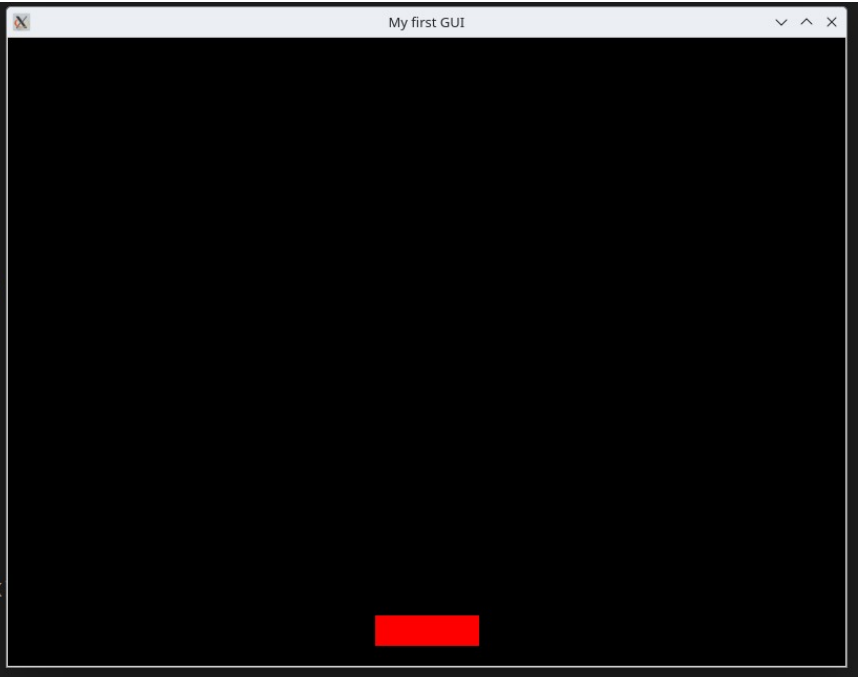
- One Spaceship (defender, simple rectangle)
- Movement (only left and right)
- Shoot-function

Motivation

- We’re going to see how multiple objects can interact on the screen :
  - Collisions
  - Movement
  - Spawning/Creating of random objects

Revisit the code

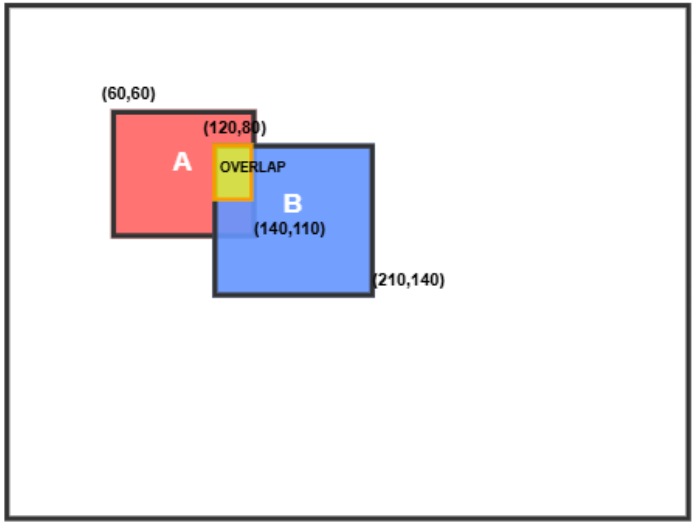
```python
import tkinter as tk
import time

def move_left(event):
    coords = canvas.coords(spaceship)
    x1, y1, x2, y2 = coords
    print ("left:", x1)
    if x1 > 0:
        canvas.move(spaceship, -10, 0)

def move_right(event):
    print ("right")
    canvas.move(spaceship, 10, 0)

def shoot(event):
    coords = canvas.coords(spaceship)
    x1, y1, x2, y2 = coords
    bullet = canvas.create_rectangle(x
    print("shoot")
    bullet_move(bullet)

def bullet_move(bullet):
    print("move")
    canvas.move(bullet, 0, -10)
```



Collision detection

- Common approaches:
  - Axis-Aligned Bounding Box – Works well for rectangles and approximations of oval
  - It uses canvas.bbox(id):
    - 2 boxes overlap when – x1_a < x2_b and x2_a > x1_b & y1_a < y2_b and y2_a>y1_b

📦 Bounding Box Collision Detection



📍 Coordinates

```
Red Box A: x1=60, y1=60, x2=140, y2=110
Blue Box B: x1=120, y1=80, x2=210, y2=140
```

🧮 Collision Check

X-axis: `60 < 210?` ✓ AND `140 > 120?` ✓
Y-axis: `60 < 140?` ✓ AND `110 > 80?` ✓

COLLISION! 💥
All 4 checks are ✓

Formula:

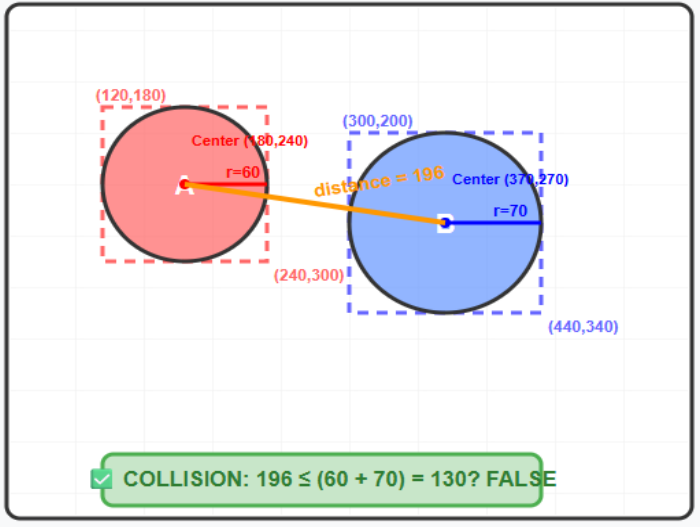
```
x1_a < x2_b AND x2_a > x1_b
y1_a < y2_b AND y2_a > y1_b
```

Collision Detection

- Another approach:
  - Circle based collision:
    - Better for round objects
    - Compute centers and radius from bbox

Circle-Based Collision Detection

Converting bbox to circles for more accurate collision detection



Step-by-Step Calculation

1 Get bounding boxes from canvas.bbox(id)

```
Circle A: (120, 180, 240, 300)
Circle B: (300, 200, 440, 340)
```

2 Calculate centers and radii

```
Center A: (120+240)/2, (180+300)/2 = (180, 240)
Center B: (300+440)/2, (200+340)/2 = (370, 270)
Radius A: (240-120)/2 = 60, Radius B: (440-300)/2 = 70
```

3 Calculate distance between centers

```
dx = 370 - 180 = 190
dy = 270 - 240 = 30
distance = √(190² + 30²) = √36,900 ≈ 196
```

4 Check collision

Rule: collision if distance ≤ (r1 + r2)

```
196 ≤ (60 + 70) = 130? ❌ FALSE → NO COLLISION
```

Advantages of Circle Collision

- ✓ More accurate for round objects (bullets, enemies, power-ups)
- ✓ Eliminates false positives from rectangle corners
- ✓ Better gameplay feel - collisions look "right" to players
- ⚠ Slightly more computation than AABB (square root calculation)

Python Implementation

```
def circle_collision(bbox1, bbox2):
  cx1, cy1 = (bbox1[0]+bbox1[2])/2, (bbox1[1]+bbox1[
  r1 = (bbox1[2] - bbox1[0]) / 2
  # Same for bbox2, then check:
  return distance ≤ (r1 + r2)
```

Legend: Bounding Box · Circle Shape · Center Point · Radius

Multiple rows of Enemies

- Instead of random spawns we would like to implement a grid formation
- To print patters the most effective tools that we can use are for loops
- Rows and columns created using nested loops is a way to implement patterns of enemies.

Multiple rows of Enemies

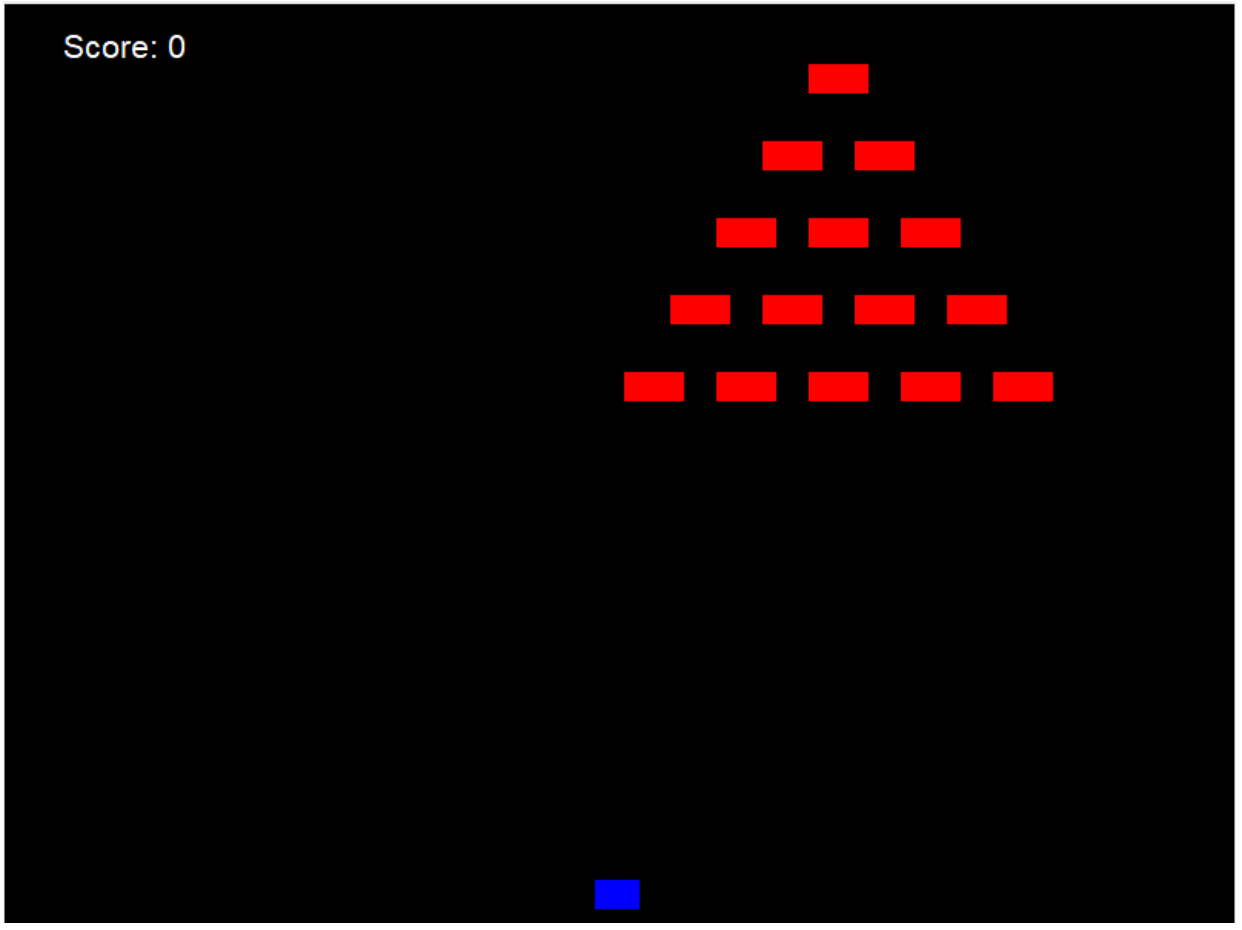
```
for row in range(1,8):
    for space in range(8-row):
        print(" ", end = "")
    for star in range(1,2*row):
        print("*", end = "")
    print()
```

Multiple rows of Enemies

```
for row in range(1,8):
    for space in range(8-row):
        print(" ", end = "")
    for star in range(1,2*row):
        print("*", end = "")
    print()
```

Example of an implementation

```
# Enemy spawn: pyramid formation
def spawn_pyramid_enemies(rows=ENEMY_START_ROWS):
    """Spawn enemies in a pyramid pattern at the top of the screen."""
    spacing_x = 60   # horizontal spacing
    spacing_y = 50   # vertical spacing
    start_y = 50

    for row in range(rows):
        count = row + 1  # number of enemies in this row
        total_width = (count - 1) * spacing_x
        start_x = WIDTH // 2 - total_width // 2

        for col in range(count):
            x = start_x + col * spacing_x
            y = start_y + row * spacing_y
            enemy = canvas.create_rectangle(x-20, y-10, x+20, y+10, fill='red')
            enemies.append(enemy)
```


Score: 0

Group Movement

- The goal here is :
  - All enemies move together (left → right)
  - When one touches the screen border :
    - Reverse the direction
    - Drop a step down

```
if x2 >= WIDTH or x1 <= 0:
    direction = -direction
    for e in enemies:
        canvas.move(e, 0, ENEMY_DROP)
```

Win and Lose Conditions

- Win : All enemies destroyed
- Lose : An enemy reaches the bottom

```
if not enemies:
    print("YOU WIN!")
if canvas.coords(enemy)[3] >= HEIGHT-40:
    print("GAME OVER")
```

Increasing Difficulty

- As score increases → make enemies faster
- Example: every 50 points, increase speed

```
if score and score % 50 == 0:
    ENEMY_SPEED += 1
```

Move to OO

- Classes
- Spaceship
  - Defender
  - Invader
- Bullet
- GameApp

OO Code

```python
class Spaceship(ABC):
    def __init__(self, canvas, x, y, width, height, color, speed=1, health=1):
        self.canvas = canvas
        self.width = width
        self.height = height
        self.color = color
        self.speed = speed
        self.health = health

        self.id = self.canvas.create_rectangle(
            x, y, x + width, y + height, fill=color
        )

        self.bullets = []

    def move_left(self):
        self.canvas.move(self.id, -self.speed, 0)

    def move_right(self):
        self.canvas.move(self.id, self.speed, 0)

    @abstractmethod
    def shoot(self):
        pass

    def update_bullets(self):
        for bullet in self.bullets:
            bullet.move()

        # Remove inactive bullets
        self.bullets = [b for b in self.bullets if b.active]
```

```python
class Bullet(ABC):
    def __init__(self, canvas, x, y, width, height, color, speed, damage):
        self.canvas = canvas
        self.width = width
        self.height = height
        self.color = color
        self.speed = speed
        self.damage = damage
        self.active = True

        # Draw bullet
        self.id = self.canvas.create_rectangle(
            x, y, x + width, y - height, fill=color
        )

    @abstractmethod
    def move(self):
        pass


class BasicBullet(Bullet):
    def move(self):
        if not self.active:
            return

        self.canvas.move(self.id, 0, -self.speed)

        # If bullet leaves screen: deactivate
        x1, y1, x2, y2 = self.canvas.coords(self.id)
        if y2 < 0:
            self.active = False
            self.canvas.delete(self.id)
```

Outline and Files

- Can be implemented using multiple files ideally. Single file app (recommended for beginners)
- Sections inside the file:
  - Imports and constants (WIDTH, HEIGHT, colors)
  - root, canvas creations
  - Player, Missile, Enemy classes (skeletons)

Random spawn & motion patterns

- Spawn pattern ideas:
  - Spawn at random X along the top
  - Random the vy (fall speed) and occasional vx to make them zig zag
  - Vary sizes/colors to indicate difficulty
- ***TASK: Implement 3 enemy types***
  - ***Slow, big (easy to hit, worth fewer points)***
  - ***Fast, small (hard to hit, worth more points)***
  - ***Zig zagging (changes vx occasionally)***